

Naive Bayes Model

[Code ▾](#)

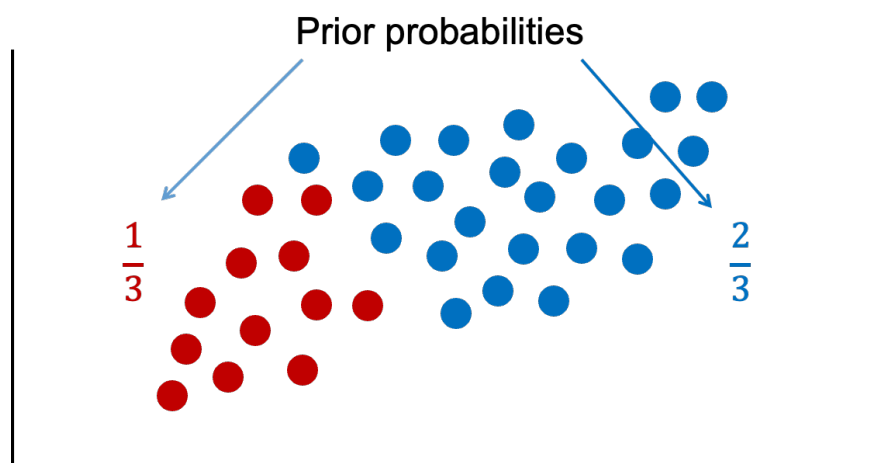
General Idea

Naïve Bayes classification is a popular algorithm for predicting a categorical target variable. When using algorithms to classify observations there are two different sources of information that we can use:

1. Similarity to other observations based on certain metrics/variables
2. Past decisions on classifications of observations like it

The first source of information is the common, frequentist approach to modeling. It predicts values of the target variable in each observation based on predictor variable values. Observations with similar predictor variable values will have similar target variable predictions. The second source of information incorporates a Bayesian approach to modeling in addition to the first source of information. This second source of information uses the information that we know about the population as a whole and previous classifications of observations.

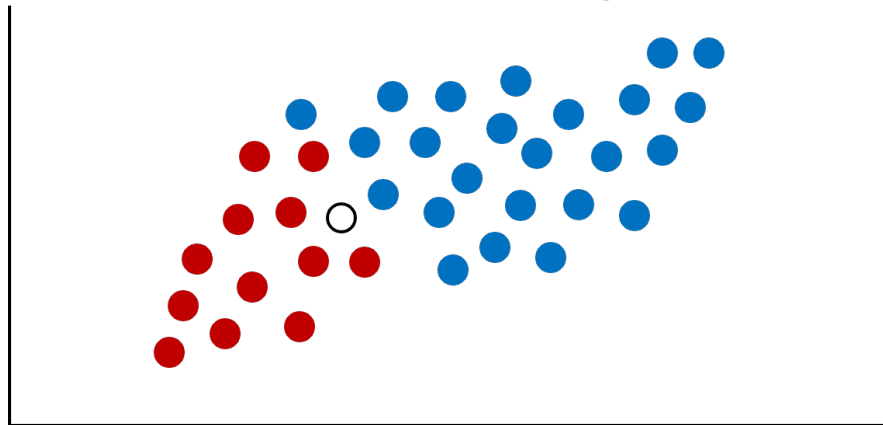
This is probably best seen through an example. Imagine we had the scatterplot of data below. In this scatterplot, we have two classes of observations we are trying to predict - red and blue.



Let's look at that second source of information - previous information about the classification. Overall in our data set, there are twice as many blues as there are reds. These probabilities are called **prior probabilities**.

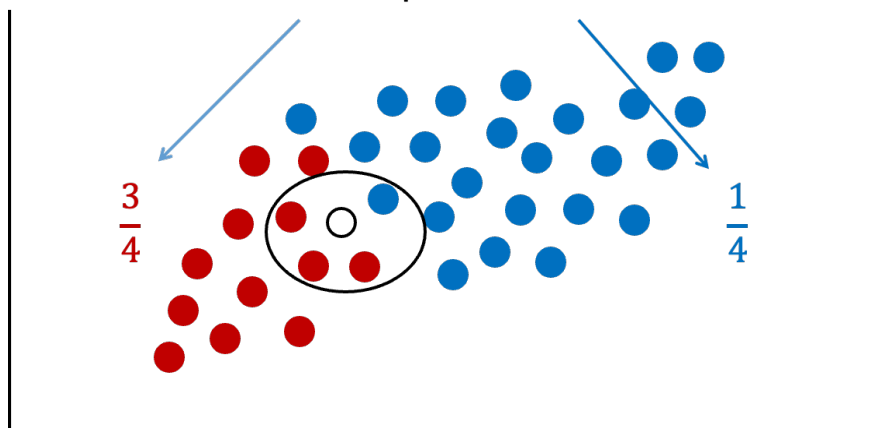
Now imagine we had a new observation that we wanted to classify.

New observation to classify.



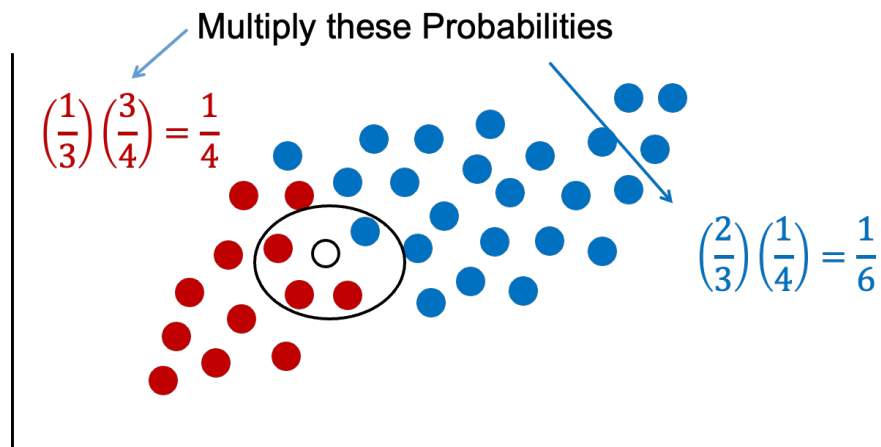
If we were to use only our previous information about the data, we would guess this new point is blue because historically we have had more blues than reds in the overall population. However, let's bring in the first piece of information - looking at observations with similar characteristics. We can define these observations visually, by looking at observations that are narrowly around the point of interest.

Conditional probabilities

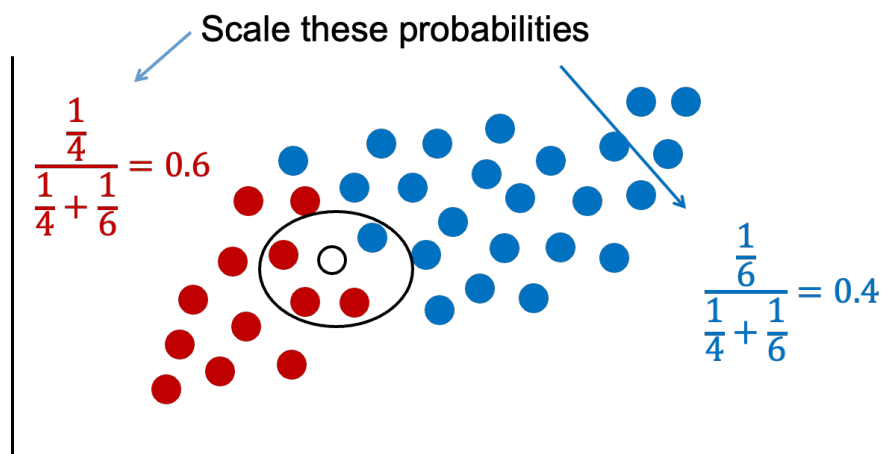


If we were only to look at observations that are similar as this new observation (the points in the oval), then we would see that there are 3 times more red observations than blue observations. These probabilities are called **conditional probabilities**. If we were to only use this information from similar observations in the data, we would guess this new point is red because there are more reds than blues that look like our new data point.

Naïve Bayes combines both of these pieces of information together. We will multiply our prior probabilities by our conditional probabilities.



The downside of this is that these probabilities are not as intuitive since they do not sum up to 1. Therefore, these probabilities are scaled to make their sum equal to 1 and their values more interpretable.



Now we have final probabilities (called **posterior probabilities**) of both red and blue for our new observation we are trying to classify. Based on the math above, there is a 60% chance the new observation is red and a 40% chance it is blue. If we only use a frequentist approach (the first source of information), we would more strongly think the new observation is red since 75% of the data that looks like that new point is red. However, the Bayesian side of the problem brings in our prior information where 67% of the overall data is blue. Our final guess is still red, but it is not as high as before because of the correction from the prior data - our second source of information.

One big assumption of the Naïve Bayes classification method is rather hard to accept - predictor variables are independent in their effects on the classification, or in other words, no interactions. This assumption is the "naïve" part of the algorithm. However, in practice, this assumption doesn't seem to drastically impact our final posterior probability predictions.

Underlying Math

Bayesian classifiers are based on Bayes' Theorem:

$$P(y|x_1, x_2, \dots, x_p) = \frac{P(y) \times P(x_1, x_2, \dots, x_p|y)}{P(x_1, x_2, \dots, x_p)}$$

The Naïve Bayes classifier assumes that the effect of the inputs are independent of one another. Remember the rule about probabilities and independent events:

$$P(A \cap B) = P(A) \times P(B)$$

Based on this rule, Bayes' Theorem now becomes:

$$P(y|x_1, x_2, \dots, x_p) = \frac{P(y) \times P(x_1|y) \times \dots \times P(x_p|y)}{P(x_1) \times \dots \times P(x_p)}$$

This makes the math much easier to calculate!

Let's work through a simple example based on the following table:

Size	Color	Accident
Large	Blue	Yes
Large	Red	Yes
Large	Blue	No
Large	Blue	No
Medium	Red	Yes
Medium	Blue	Yes
Medium	Red	Yes
Small	Blue	No
Small	Red	Yes
Small	Red	No

We will try to predict the probability of getting into an accident based on two variables - size of car and color of car.

Imagine we had a new observation that is a blue, medium car. Let's use Bayes' Theorem to calculate the probabilities of a Yes and No for accident:

$$P(Y|M\&B) = \frac{P(Y) \times P(M|Y) \times P(B|Y)}{P(M) \times P(B)}$$

From the table above we can see that there are 6 out of 10 cars that get into an accident, $P(Y) = 0.6$. Of the cars that get into an accident, 3 of the 6 are medium, $P(M|Y) = 0.5$. Of the cars that get into an accident, 2 of the 6 are blue, $P(B|Y) = 0.333$. Of all of the cars, 3 out of the 10 are medium, $P(M) = 0.3$, and 5 out of 10 are blue, $P(B) = 0.5$. Inputting these values into the equation above, we get a probability of getting into an accident given the car is blue and medium as $P(Y|M\&B) = 0.667$.

Let's do the same thing for the probability of not getting into an accident given the car is blue and medium:

$$P(N|M\&B) = \frac{P(N) \times P(M|N) \times P(B|N)}{P(M) \times P(B)}$$

We can see that 4 out of the 10 cars did **not** get into an accident, $P(N) = 0.4$. Of the cars that did not get into an accident, 0 out of the 4 were medium sized, $P(M|N) = 0$. This poses a problem. With this 0 in the calculation, the probability of not getting into an accident is forced to 0. This is more likely due to our small sample size and not truly representative of the population as a whole. This is similar to the problem of quasi-complete separation in logistic regression. Luckily, the Naïve Bayes algorithm has a built in mechanism to handle this. The algorithm uses a Laplace correction, essentially adding a small constant to each of the counts to prevent any 0 probability calculations.

	Yes	No		Yes	No
Small	1	2	Small	1.01	2.01
Medium	3	0	Medium	3.01	0.01
Large	2	2	Large	2.01	2.01

For example, instead of a classification table comparing size of car to accident (yes or no) as our original data has it (the left table above), the algorithm instead will add a small constant to each cell (the right table above). Now the calculation for the probability of a medium car given no accident becomes $P(M|N) = 0.01/4.03 = 0.0025$. Now we can fill out the rest easily, $P(B|N) = 0.75$, $P(M) = 0.3$, and $P(B) = 0.5$. Inputting these values into the equation above, we get a probability of not getting into an accident given the car is blue and medium as 0.667, $P(N|M\&B) = 0.005$.

Remember, these are not scaled. If we scale these probabilities, our probability of getting into an accident given the car is blue and medium is 0.993. The probability of not getting into an accident is now 0.007.

Fitting Naive Bayes

We worked through the Naïve Bayes algorithm when we had a categorical

target and categorical predictor variable. In this situation, we determine the predicted probability of each target category based on cross-tabulation tables of each variable with the target variable (same idea as previous section). However, when we have numeric predictor variables, the process is a little different. With a continuous predictor variable, the algorithm determines the probability on either values from a Normal (Gaussian) distribution with the same mean and standard deviation as our data or a kernel density estimation of the data.

Although the Naïve Bayes classifier was designed for target variables that are categorical, some softwares can also apply the algorithm to continuous target variables as well. In these situations, the software actually treats the continuous target variable as a categorical variable with a large number of categories. The value of the target variable that is the highest probability will be the prediction for the continuous target variable.

Let's see this in both of our softwares!

R Python

In R we can have either a continuous or categorical target variable. Let's see how to use R for each.

With a continuous target variable, we can just use the `naiveBayes` function from the `e1071` package. The `usekernel = TRUE` tells the function to use a kernel density estimation for continuous predictor variables instead of a normal distribution. Again, the function will treat the continuous target variable as a categorical variable with a large number of categories.

▼ Code

```
library(e1071)

set.seed(12345)
nb.ames <- naiveBayes(Sale_Price ~ ., data = traini
```

If we wanted to tune the Naïve Bayes model parameters, we will need to use the `train` function from the `caret` package. However, the `train` function will only apply the Naïve Bayes classifier to a categorical target variable. We will create a categorical target variable called `Bonus` that imagines homes selling for more than \$175,000 nets the real estate agent a bonus. If `Bonus` takes a value of 1, the house sold for more than \$175,000 and 0 otherwise.

▼ Code

```
ames <- ames %>%
  mutate(Bonus = ifelse(Sale_Price > 175000, 1, 0))
```

```

set.seed(4321)

training_c <- ames %>% sample_frac(0.7)
testing_c <- anti_join(ames, training_c, by = 'id')

training_c <- training_c %>%
  select(Bonus,
         Bedroom_AbvGr,
         Year_Built,
         Mo_Sold,
         Lot_Area,
         Street,
         Central_Air,
         First_Flr_SF,
         Second_Flr_SF,
         Full_Bath,
         Half_Bath,
         Fireplaces,
         Garage_Area,
         Gr_Liv_Area,
         TotRms_AbvGrd)

```

In the `train` function we will tune 3 parameters in the `expand.grid`. We will allow the algorithm to use either a normal distribution or a kernel distribution with the `usekernel` option. The other tuning parameters are the Laplace correction (`fL`) and bandwidth adjustment (`adjust`).

▼ Code

```

library(caret)
library(klaR)

tune_grid <- expand.grid(
  usekernel = c(TRUE, FALSE),
  fL = c(0, 0.5, 1),
  adjust = c(0.1, 0.5, 1)
)

set.seed(12345)
nb.ames.caret <- caret::train(factor(Bonus) ~ ., data = ames,
                              method = "nb",
                              tuneGrid = tune_grid,
                              trControl = trainControl(method = "cv",
                                                         number = 10))

nb.ames.caret$bestTune

```

```

      fL usekernel adjust
11    0         TRUE    0.5

```

From the output above, the best Naïve Bayes algorithm has a Laplace correction value of 0, a bandwidth adjustment of 0.5, and uses the

kernel distributions for continuous predictor variables.

Summary

In summary, Naïve Bayes models are good models to use for prediction, but explanation becomes more difficult and complex. Some of the advantages of using Naïve Bayes:

- Very simple to implement
- Good at predictions (especially good classification for few categories)
- Performs best with categorical variables / text
- Fast computational time
- Robust to noise and irrelevant variables

There are some disadvantages though:

- Independence assumption
- Careful about normality assumption for continuous variables
- Requires more memory storage than most models
- Trust predicted categories more than probabilities